

Trabajo Práctico 2 — NAT

[75.43] Introducción a los Sistemas Distribuidos

Curso Martes, Grupo 10

Primer cuatrimestre de 2026

Alumno	Número de padrón	Email
MORTIMER, Federica	108034	fmortimer@fi.uba.ar
MALDONADO, Oswaldo	110404	omaldonado@fi.uba.ar
OUTEIRO, Sebastian	92018	souteiro@fi.uba.ar
CONTI, Tomás Bautista	111760	tconti@fi.uba.ar
MAYDANA, Lionel	106512	lmaydana@fi.uba.ar

Índice

1. Descripción general de la solución	2
2. Diseño de la tabla NAT	2
3. Manejo de ARP	3
4. Estrategia de instalación de flujos	3
5. Decisiones de diseño	4
6. Problemas encontrados y soluciones implementadas	5
7. Pruebas realizadas	5
7.1. Conectividad básica	5
7.2. Múltiples clientes	13
7.3. Diferentes tipos de tráfico	16
8. Conclusión	16

1. Descripción general de la solución

El presente trabajo implementa un **NAT saliente con traducción por puertos (PAT)** sobre una arquitectura de Redes Definidas por Software (SDN). La red está compuesta por un switch OpenFlow (S1) controlado remotamente por POX, que actúa como dispositivo de capa 3 con dos interfaces: una pública (200.0.0.254) y una privada (192.168.1.254).

El controlador POX concentra toda la lógica de decisión (*plano de control*): resolución ARP, traducción de direcciones IP y puertos, e instalación de reglas OpenFlow. Una vez instaladas las reglas, el switch procesa de forma autónoma el tráfico subsiguiente de cada conexión (*plano de datos*), minimizando la intervención posterior del controlador.

El flujo general de una conexión saliente es el siguiente:

1. Un host privado envía un paquete. El switch no posee una regla que haga match y lo eleva al controlador.
2. El controlador resuelve la dirección MAC del destino mediante ARP. Si no la conoce, encola el paquete hasta obtener la respuesta.
3. El controlador asigna un puerto público NAT, instala dos flujos (saliente y entrante) en el switch y reenvía el paquete actual ya traducido.
4. Los paquetes siguientes de esa conexión son procesados directamente por el switch, sin volver a pasar por el controlador.

2. Diseño de la tabla NAT

Por indicación de la cátedra, **no se implementa una tabla de traducción explícita**. El estado de cada conexión activa reside en los flujos OpenFlow instalados en el switch. El controlador mantiene únicamente:

- **used_nat_ports**: conjunto de puertos públicos actualmente en uso.
- **next_nat_port**: puntero al próximo puerto a asignar, dentro del rango [1024,65535] con *wrap-around*.

Esto alcanza para garantizar la unicidad de las traducciones y la reutilización de puertos. El par de flujos saliente/entrante instalado por cada conexión cumple la función de “entrada de tabla”: el flujo entrante utiliza el puerto público NAT como discriminador para reenviar el tráfico de retorno al host privado correcto.

El esquema conceptual de la traducción es:

Protocolo	IP privada	Puerto privado	IP pública	Puerto público
TCP/UDP	192.168.1.2	5000	200.0.0.254	1024
TCP/UDP	192.168.1.3	5000	200.0.0.254	1025
TCP/UDP	192.168.1.2	5001	200.0.0.254	1026

Cuadro 1: Esquema conceptual de la traducción PAT

Cuando el flujo entrante expira por *timeout*, el switch notifica al controlador mediante un evento **FlowRemoved** y este libera el puerto NAT correspondiente, dejándolo disponible para nuevas conexiones.

3. Manejo de ARP

El controlador implementa ARP dinámico completo, sin entradas estáticas en ningún host de la topología.

Tabla ARP dinámica. Se mantiene un diccionario `arp_table` ($IP \rightarrow (MAC, \text{timestamp})$) que se actualiza ante cualquier paquete ARP recibido, tanto *Request* como *Reply*.

ARP Request recibido. Si está dirigido a una de las IP del router (192.168.1.254 o 200.0.0.254), el controlador responde con la MAC correspondiente (00:00:00:bb:bb:bb para la interfaz privada, 00:00:00:aa:aa:aa para la pública). Si no está dirigido al router, se ignora.

ARP Reply recibido. Se aprende la MAC del emisor y se procesan todos los paquetes que estaban encolados (`pending_packets`) en espera de esa resolución.

ARP Request generado. Cuando el controlador necesita reenviar un paquete hacia la red pública y desconoce la MAC del destino, encola el paquete en `pending_packets` y emite un ARP Request en *broadcast* por el **puerto público**, usando la IP y MAC públicas del router como origen. La resolución activa, por lo tanto, se realiza únicamente del lado público (el servidor). Las MAC de los hosts privados no se resuelven con un ARP Request propio: se aprenden de forma pasiva, ya sea del ARP Request que el host envía a su gateway o de la dirección de origen del primer paquete IP que envía. Esto es suficiente porque, al tratarse de un NAT saliente, el host privado siempre inicia la conexión y su MAC ya es conocida cuando se necesita instalar el flujo de retorno.

El flujo completo para un host privado que inicia una conexión es:

1. El host privado emite un ARP Request por 192.168.1.254; el controlador responde con 00:00:00:bb:bb:bb.
2. El host envía el paquete IP. El controlador desconoce la MAC del servidor público, por lo que encola el paquete y emite un ARP Request en broadcast.
3. El servidor responde el ARP. El controlador aprende su MAC, desencola el paquete y lo procesa.

4. Estrategia de instalación de flujos

La instalación de flujos es **reactiva**: el controlador solo interviene ante el primer paquete de una conexión (o ante eventos de ARP y expiración de flujos). Por cada nueva conexión TCP/UDP se instalan **dos flujos** en el switch.

Flujo saliente (red privada $\rightarrow$ red pública).

- *Match*: `nw_src` = IP privada, `in_port` = puerto del host, `dl_type` = IPv4, `nw_proto` = TCP/UDP, `tp_src` = puerto privado.

- *Acciones* (en orden): reescribir IP origen a 200.0.0.254, reescribir puerto origen al puerto NAT asignado, reescribir MAC origen a 00:00:00:aa:aa:aa, reescribir MAC destino a la del servidor, y enviar por el puerto público.

Flujo entrante (red pública → red privada).

- *Match*: `nw_src` = IP del servidor, `nw_dst` = 200.0.0.254, `in_port` = puerto público, `dl_type` = IPv4, `nw_proto` = TCP/UDP, `tp_dst` = puerto NAT.
- *Flag*: `OFPPF_SEND_FLOW_REM`, para que el switch notifique al controlador cuando el flujo expire.
- *Acciones* (en orden): reescribir IP destino a la IP privada, reescribir puerto destino al puerto privado original, reescribir MAC destino a la del host privado, reescribir MAC origen a 00:00:00:bb:bb:bb, y enviar por el puerto del switch correspondiente al host privado.

Timeouts. Ambos flujos utilizan un `idle_timeout` de 10 segundos. Al expirar el flujo entrante, el evento `FlowRemoved` libera el puerto NAT en el controlador, garantizando la liberación de recursos.

5. Decisiones de diseño

Estado implícito en los flujos. No se mantiene una tabla NAT explícita en el controlador. Cada par de flujos instalados es la entrada de la tabla. Esto reduce el uso de memoria del controlador y evita inconsistencias de sincronización entre el estado del controlador y el del switch.

Asignación de puertos con *wrap-around*. El puntero `next_nat_port` avanza secuencialmente y, al alcanzar el valor 65535, vuelve a 1024. Si todos los puertos están ocupados, se lanza un error. Este esquema permite una asignación eficiente en el caso común sin requerir búsquedas costosas.

Liberación de puertos vía `FlowRemoved`. En lugar de emplear temporizadores propios en el controlador, se delega la detección de expiración al switch mediante el flag `OFPPF_SEND_FLOW_REM`. Esto mantiene la coherencia entre el estado del switch y el del controlador.

Soporte multi-host. El *match* del flujo saliente incluye la IP de origen del host privado, por lo que múltiples hosts coexisten con entradas independientes. El flujo entrante emplea el puerto NAT como discriminador único para determinar a qué host privado reenviar el tráfico de retorno.

Reutilización de puertos. Un puerto liberado por expiración vuelve a estar disponible inmediatamente en `used_nat_ports` para conexiones futuras.

Encolado durante la resolución ARP. Si la MAC del destino se desconoce, el paquete se almacena en `pending_packets` y se procesa en orden una vez recibido el ARP Reply, evitando descartar tráfico legítimo.

6. Problemas encontrados y soluciones implementadas

Entradas obsoletas en la tabla ARP del controlador

Dado que la tabla ARP del controlador se actualiza únicamente al recibir tráfico ARP proveniente del nodo en cuestión, una entrada podría permanecer indefinidamente si dicho nodo no genera nuevo tráfico ARP. Esto representa un problema: si la MAC asociada a una IP cambia, el controlador continuaría instalando flujos con la MAC antigua, causando pérdida de tráfico.

Para mitigar esto, cada entrada de la `arp_table` se almacena junto con un *timestamp*. Las entradas que superen un umbral de antigüedad son invalidadas, forzando una nueva resolución ARP antes de instalar cualquier flujo nuevo hacia esa dirección. Así se garantiza que un cambio de MAC sea detectado oportunamente sin requerir intervención manual.

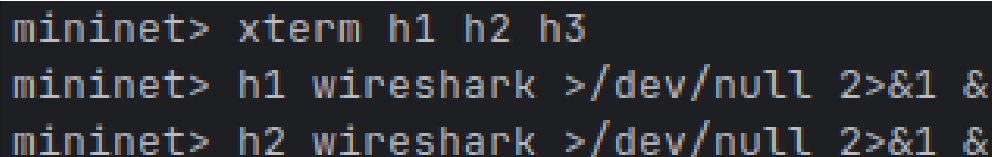
7. Pruebas realizadas

7.1. Conectividad básica

Se realizaron dos pruebas de conectividad TCP utilizando `iperf` entre H2 (cliente, red privada) y H1 (servidor, red pública), con el controlador POX gestionando la traducción NAT.

Configuración del entorno

La topología se levantó en Mininet abriendo terminales para H1, H2 y H3, y lanzando Wireshark en los nodos de interés.

A screenshot of a terminal window with a dark background and light-colored text. It shows three lines of commands entered in a Mininet environment. The first line opens three xterm windows for hosts h1, h2, and h3. The second and third lines start Wireshark on hosts h1 and h2 respectively, with output redirected to /dev/null and backgrounded.

```
mininet> xterm h1 h2 h3
mininet> h1 wireshark >/dev/null 2>&1 &
mininet> h2 wireshark >/dev/null 2>&1 &
```

Figura 1: Configuración inicial en Mininet: apertura de terminales y lanzamiento de Wireshark.

Resultados de iperf

Se ejecutaron dos conexiones TCP consecutivas desde H2 hacia H1 (puerto 5001). Los resultados observados en H1 se muestran en la Figura 2 y los de H2 en la Figura 3.

```

"Node: h1"
root@dell15450-so:/home/sebastianouteiro/GIT/Sebas/FIUBA/redes/tp2-redes# h1 wir
eshark >/dev/null 2>&1 &
[1] 37626
root@dell15450-so:/home/sebastianouteiro/GIT/Sebas/FIUBA/redes/tp2-redes# iperf
-s

-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 1024
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-9.8454 sec 37.0 GBytes 32.3 Gbits/sec
[ 2] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 1025
[ ID] Interval      Transfer    Bandwidth
[ 2] 0.0000-10.0004 sec 41.4 GBytes 35.6 Gbits/sec

```

Figura 2: Resultados de `iperf` en H1 (servidor). Se observan dos conexiones entrantes desde 200.0.0.254 con puertos NAT 1024 y 1025.

```

"Node: h2"
root@dell15450-so:/home/sebastianouteiro/GIT/Sebas/FIUBA/redes/tp2-redes# iperf
-c 200.0.0.1

-----
Client connecting to 200.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.1.2 port 45754 connected with 200.0.0.1 port 5001
^C[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-9.8568 sec 37.0 GBytes 32.3 Gbits/sec
root@dell15450-so:/home/sebastianouteiro/GIT/Sebas/FIUBA/redes/tp2-redes# iperf
-c 200.0.0.1

-----
Client connecting to 200.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.1.2 port 58866 connected with 200.0.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0056 sec 41.4 GBytes 35.6 Gbits/sec
root@dell15450-so:/home/sebastianouteiro/GIT/Sebas/FIUBA/redes/tp2-redes#

```

Figura 3: Resultados de `iperf` en H2 (cliente). Los puertos locales utilizados fueron 45754 y 58866.

Pueden observarse los siguientes puntos:

- La IP de origen recibida por H1 en ambas conexiones fue 200.0.0.254, es decir, la IP pública del router, lo cual confirma que el NAT saliente funciona correctamente.
- Los puertos asignados por el controlador fueron 1024 y 1025, mientras que H2 utilizó internamente los puertos 45754 y 58866 respectivamente. H1 nunca observó los puertos privados de H2.

Logs del controlador POX

La Figura 4 muestra los logs generados por el controlador durante ambas pruebas TCP.

```

INFO:protorouter:ARP REQUEST recibido de 192.168.1.2 (00:00:00:00:00:02) para 192.168.1.254 | In Port: 2
INFO:protorouter:ARP - Entrada Agregada: 192.168.1.2 + 00:00:00:00:00:02
INFO:protorouter:ARP Reply enviado: 192.168.1.254 está en 00:00:00:bb:bb:bb + Out Port: 2
INFO:protorouter:Paquete IP RECIBIDO: 192.168.1.2 + 200.0.0.1 | MAC: 00:00:00:00:00:02 + 00:00:00:bb:bb:bb | In Port: 2
INFO:protorouter:Protocol 6 | src_port: 45754 + dst_port: 5001
INFO:protorouter:MATCH: 192.168.1.2 pertenece a la red privada 192.168.1.0/24
INFO:protorouter:ARP Request enviado: ¿Quién tiene 200.0.0.1? + port 1
INFO:protorouter:MAC de 200.0.0.1 desconocida. Paquete encolado, ARP Request enviado.
INFO:protorouter:ARP REPLY recibido de 200.0.0.1 (00:00:00:00:00:01) para 200.0.0.254 | In Port: 1
INFO:protorouter:ARP - Entrada Agregada: 200.0.0.1 + 00:00:00:00:00:01
INFO:protorouter:Procesando 1 paquete(s) pendiente(s) para 200.0.0.1
INFO:protorouter:Puerto NAT asignado: 1024 | En uso: 1
INFO:protorouter:PAT TCP: 192.168.1.2:45754 + 200.0.0.1:1024
INFO:protorouter:ENVIANDO: 192.168.1.2 + 200.0.0.1 | NAT IP: 200.0.0.254 | MAC: 00:00:00:aa:aa:aa + 00:00:00:00:00:01 | Out Port: 1
INFO:protorouter:ARP REQUEST recibido de 200.0.0.1 (00:00:00:00:00:01) para 200.0.0.254 | In Port: 1
INFO:protorouter:ARP - Entrada Agregada: 200.0.0.1 + 00:00:00:00:00:01
INFO:protorouter:ARP Reply enviado: 200.0.0.254 está en 00:00:00:aa:aa:aa + Out Port: 1
INFO:protorouter:Puerto NAT liberado: 1024 | En uso: 0
INFO:protorouter:Paquete IP RECIBIDO: 192.168.1.2 + 200.0.0.1 | MAC: 00:00:00:00:00:02 + 00:00:00:bb:bb:bb | In Port: 2
INFO:protorouter:Protocol 6 | src_port: 58866 + dst_port: 5001
INFO:protorouter:MATCH: 192.168.1.2 pertenece a la red privada 192.168.1.0/24
INFO:protorouter:ARP - Entrada expirada y eliminada: 200.0.0.1
INFO:protorouter:ARP Request enviado: ¿Quién tiene 200.0.0.1? + port 1
INFO:protorouter:ARP expirado para 200.0.0.1. Paquete encolado, ARP Request enviado.
INFO:protorouter:ARP REQUEST recibido de 200.0.0.1 (00:00:00:00:00:01) para 200.0.0.254 | In Port: 1
INFO:protorouter:ARP - Entrada Agregada: 200.0.0.1 + 00:00:00:00:00:01
INFO:protorouter:Procesando 1 paquete(s) pendiente(s) para 200.0.0.1
INFO:protorouter:Puerto NAT asignado: 1025 | En uso: 1
INFO:protorouter:PAT TCP: 192.168.1.2:58866 + 200.0.0.1:1025
INFO:protorouter:ENVIANDO: 192.168.1.2 + 200.0.0.1 | NAT IP: 200.0.0.254 | MAC: 00:00:00:aa:aa:aa + 00:00:00:00:00:01 | Out Port: 1
INFO:protorouter:ARP REQUEST recibido de 200.0.0.1 (00:00:00:00:00:01) para 200.0.0.254 | In Port: 1
INFO:protorouter:ARP - Entrada Agregada: 200.0.0.1 + 00:00:00:00:00:01
INFO:protorouter:ARP Reply enviado: 200.0.0.254 está en 00:00:00:aa:aa:aa + Out Port: 1
INFO:protorouter:Puerto NAT liberado: 1025 | En uso: 0

```

Figura 4: Logs del controlador POX durante las dos pruebas TCP. Se aprecian la resolución ARP, la asignación de puertos NAT y la liberación al finalizar cada conexión.

El flujo de eventos observado fue el siguiente:

1. H2 emite un ARP Request hacia 192.168.1.254; el controlador aprende la dirección de H2 y responde con la MAC privada del router (00:00:00:bb:bb:bb).
2. El controlador recibe el paquete IP de H2 (TCP SYN). Como desconoce la MAC de H1, encola el paquete y emite un ARP Request en broadcast por el puerto público.
3. H1 responde el ARP; el controlador aprende su MAC pública y desencola el paquete pendiente.
4. Se asigna el puerto NAT 1024, se instalan los flujos saliente y entrante en el switch, y se reenvía el paquete.
5. H1 emite un ARP Request hacia 200.0.0.254; el controlador responde con la MAC pública del router (00:00:00:aa:aa:aa).
6. Al finalizar la conexión, OpenFlow notifica la expiración del flujo entrante y el controlador libera el puerto 1024.
7. La segunda prueba repite el mismo flujo asignando el puerto 1025. Dado que el timeout de las entradas es de 10 segundos, entre ambas pruebas vencieron las entradas ARP y todo el intercambio ARP se repitió.

En todas las pruebas se verificó que únicamente los primeros paquetes ARP y el primer paquete IP de cada conexión pasaron por el controlador. El tráfico subsiguiente fue procesado directamente por el switch.

Capturas Wireshark

Las capturas de Wireshark en H2 (Figuras 5 y 6) y en H1 (Figuras 7 y 8) permiten verificar el comportamiento del NAT a nivel de paquete.

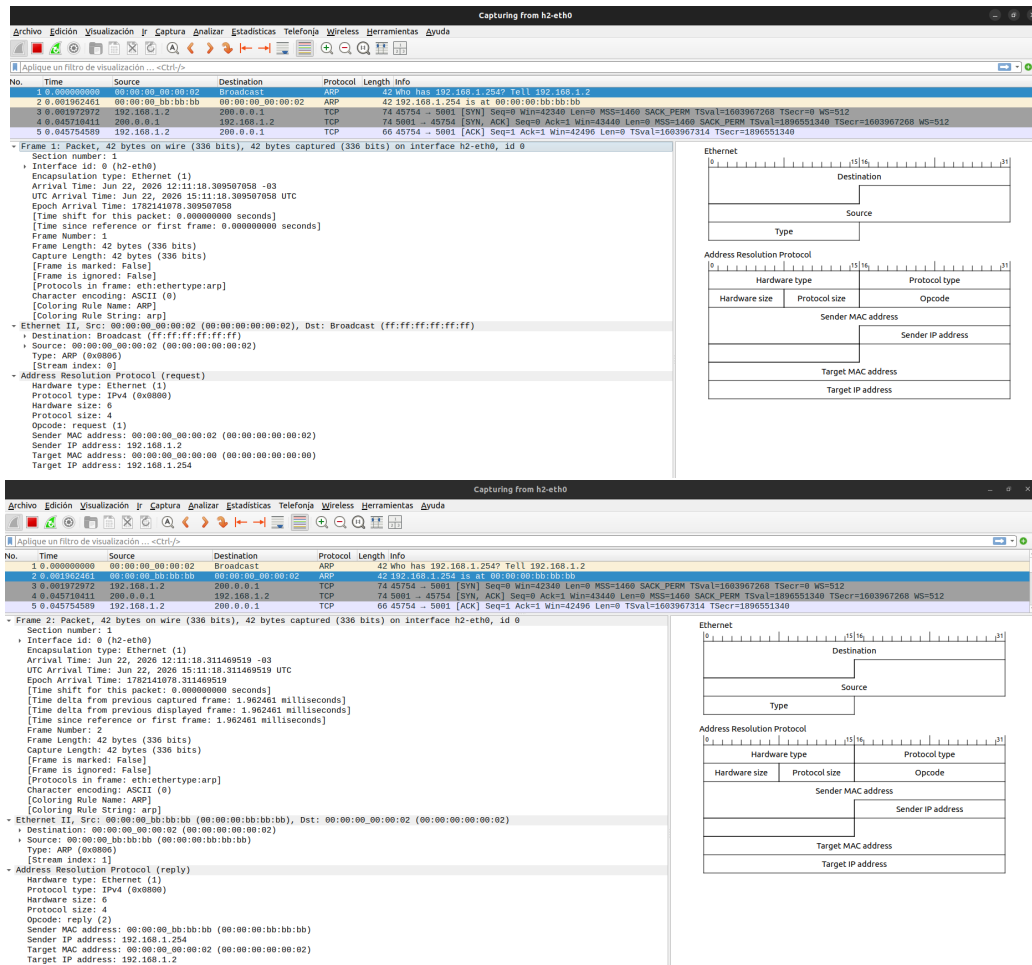


Figura 5: Wireshark en H2: secuencia de paquetes. (1) ARP broadcast hacia el router; (2) ARP reply con la MAC privada del router; (3) TCP SYN hacia H1; (4) TCP SYN-ACK recibido desde H1.

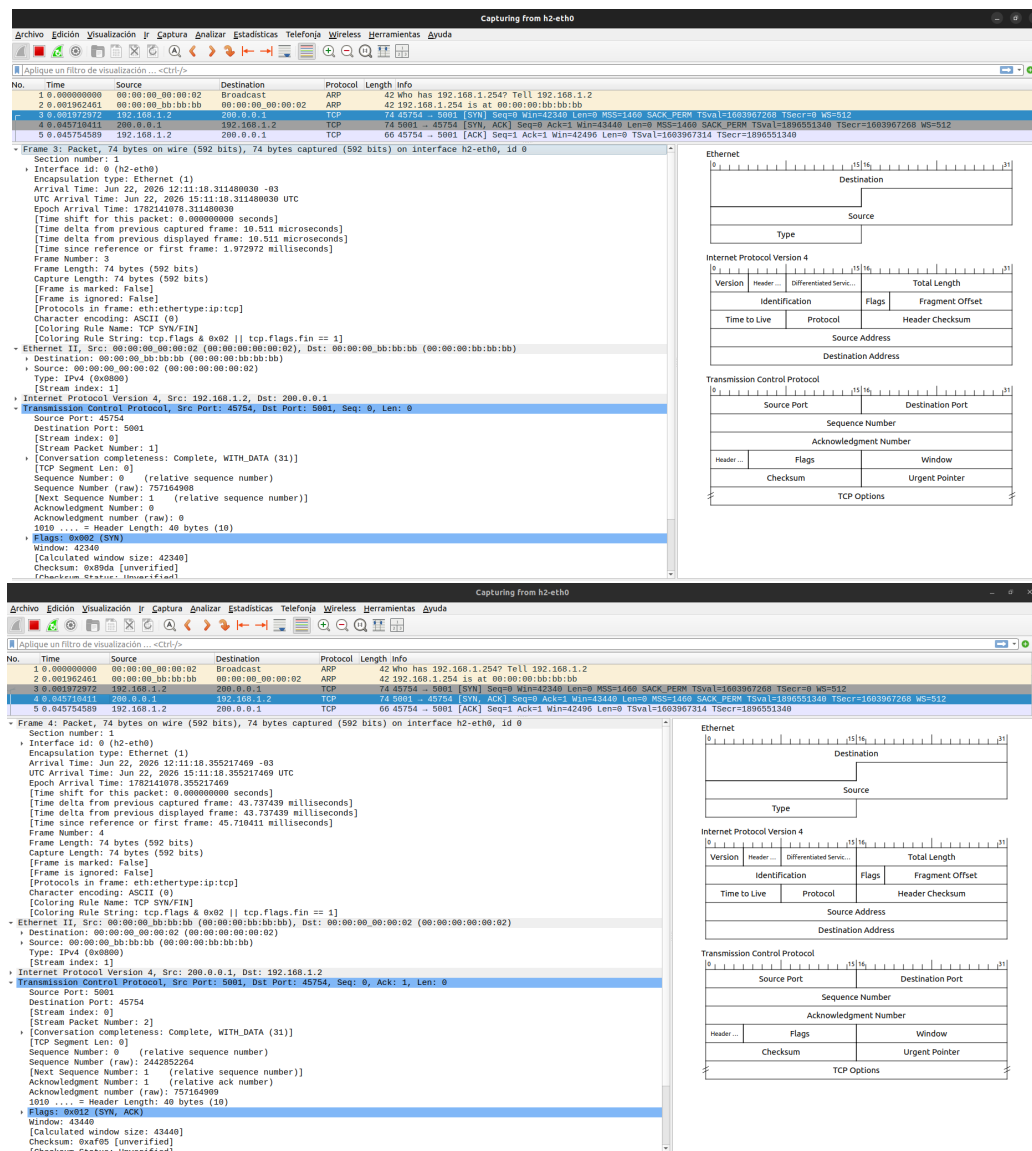


Figura 6: Wireshark en H2: detalle del paquete SYN (puerto origen 45754, destino 5001) y del SYN-ACK de respuesta.

Desde la perspectiva de H2, la conexión se establece normalmente hacia 200.0.0.1:5001 usando su puerto local 45754. H2 no percibe ninguna traducción de direcciones.

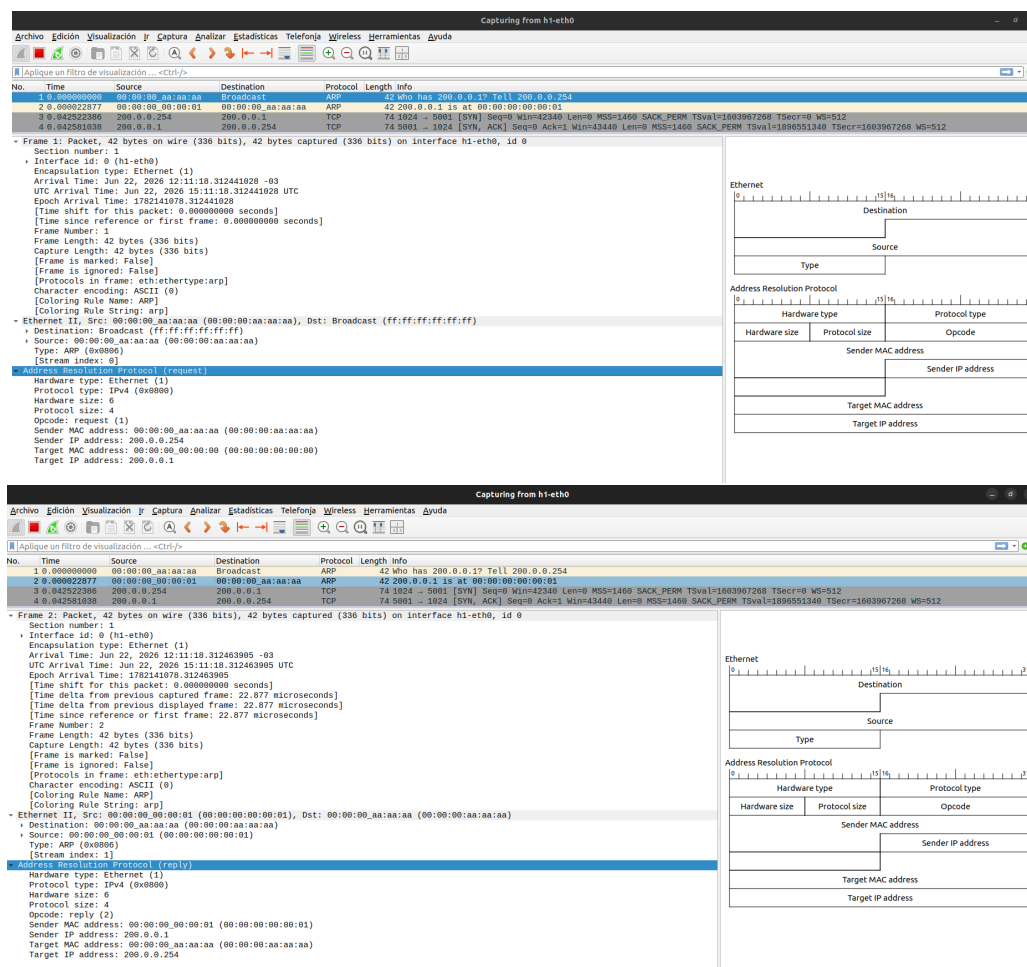


Figura 7: Wireshark en H1: secuencia de paquetes. (1) ARP Request desde la MAC pública del router preguntando por H1; (2) ARP Reply de H1; (3) TCP SYN con IP origen 200.0.0.254 y puerto 1024; (4) TCP SYN-ACK de respuesta.

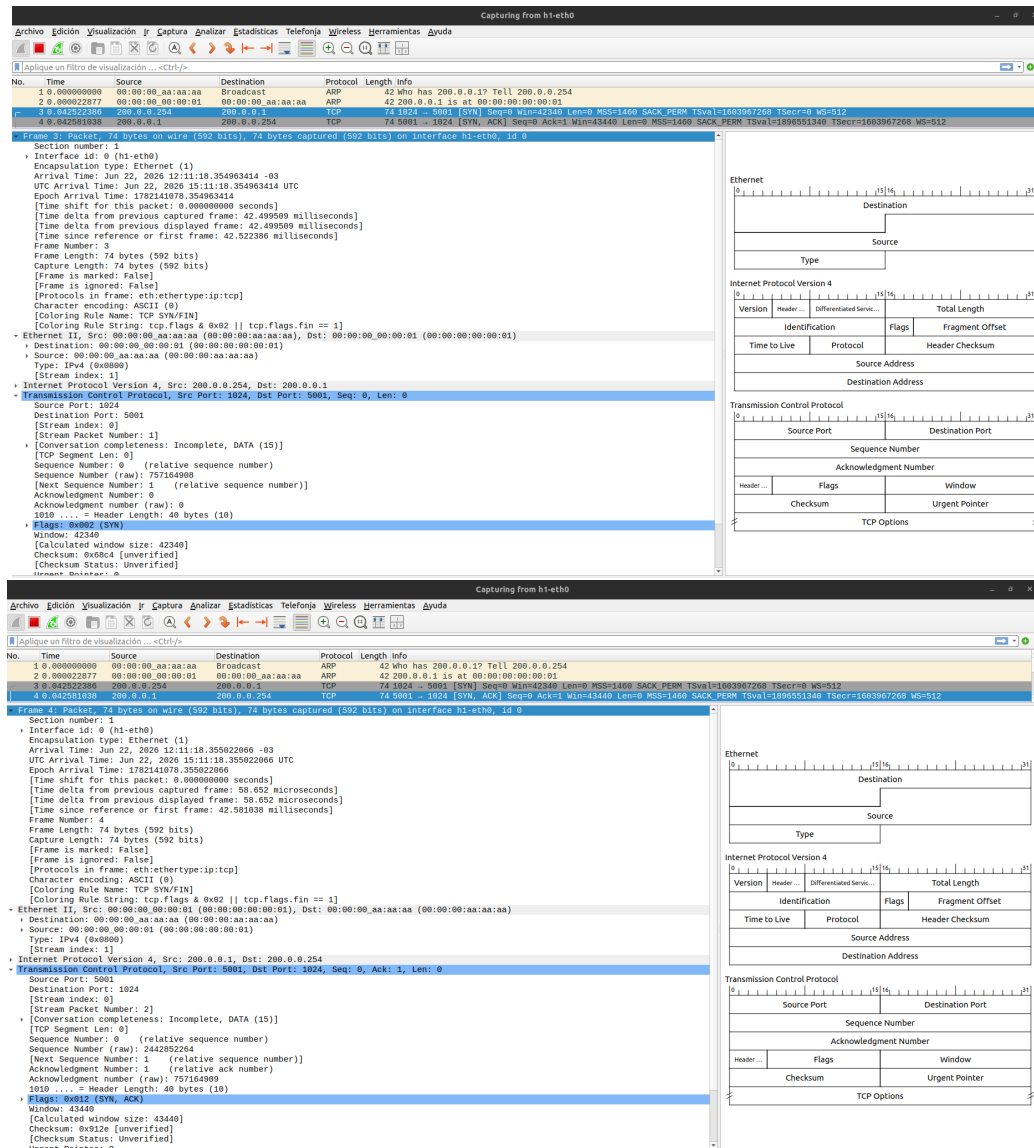


Figura 8: Wireshark en H1: detalle del paquete SYN (puerto origen NAT 1024, destino 5001) y del SYN-ACK. Confirma que el NAT se aplicó correctamente.

Desde la perspectiva de H1, la conexión proviene de 200.0.0.254:1024, es decir, de la IP pública del router con el puerto NAT asignado. H1 desconoce la existencia de H2 ni de su puerto privado.

Dump de flujos OpenFlow

Al finalizar la segunda prueba TCP se ejecutó `sudo ovs-ofctl dump-flows s1`, obteniendo los dos flujos instalados por el controlador:

1. Flujo Saliente ($h2 \rightarrow h1$)

Este flujo se encarga de interceptar los paquetes que salen desde el host interno $h2$ hacia el servidor externo $h1$, aplicando técnicas de traducción de direcciones de red y puertos.

- **Match:** `in_port=s1-eth2, nw_src=192.168.1.2, tp_src=58866`

- **Acciones:** mod_nw_src:200.0.0.254, mod_tp_src:1025, mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst:00:00:00:00:00:01, output:s1-eth1

Matches	Descripción
tcp	Solo aplica para flujos orientados a la transmisión mediante el protocolo TCP.
in_port="s1-eth2"	Indica que el paquete ingresa por el puerto 2 del switch, donde se encuentra conectado <i>h2</i> .
nw_src=192.168.1.2	Representa la dirección IP privada de origen (perteneciente a <i>h2</i>).
tp_src=58866	Representa el puerto de origen que abrió <i>h2</i> para esta conexión TCP específica.

Acciones	Descripción
mod_nw_src:200.0.0.254	Modifica la IP de origen por 200.0.0.254, la cual actúa como la IP pública del router.
mod_tp_src:1025	Modifica el puerto de origen al 1025, aplicando correctamente la traducción de puertos (PAT).
mod_dl_src:00:00:00:aa:aa:aa	Modifica la dirección MAC de origen utilizando la dirección MAC pública de la interfaz del router.
mod_dl_dst:00:00:00:00:00:01	Cambia la dirección MAC de destino (que originalmente correspondía al router) por la dirección MAC del servidor (<i>h1</i>).
output:"s1-eth1"	Envía el paquete transformado a través del puerto 1 del switch, donde está interconectado <i>h1</i> .

2. Flujo Entrante ($h2 \leftarrow h1$)

Este flujo gestiona el tráfico de retorno (las respuestas) enviado por el servidor externo *h1* hacia el host privado *h2*, realizando el proceso inverso de de-traducción.

- **Match:** in_port=s1-eth1, nw_src=200.0.0.1, nw_dst=200.0.0.254, tp_dst=1025
- **Acciones:** mod_nw_dst:192.168.1.2, mod_tp_dst:58866, mod_dl_dst:00:00:00:00:00:02, mod_dl_src:00:00:00:bb:bb:bb, output:s1-eth2

Matches	Descripción
tcp	Al igual que el anterior, restringe el match únicamente a paquetes del protocolo TCP.
in_port="s1-eth1"	Informa que el paquete de respuesta ingresa por el puerto 1 del switch, proveniente de <i>h1</i> .
nw_src=200.0.0.1	Representa la dirección IP pública de origen del servidor externo (<i>h1</i>).
nw_dst=200.0.0.254	Evalúa que la IP de destino sea la IP pública de nuestro router.
tp_dst=1025	Comprueba el puerto de destino mapeado previamente por PAT en el controlador.

Acciones	Descripción
<code>mod_nw_dst:192.168.1.2</code>	Restaura la dirección IP de destino original cambiándola por la IP privada de <i>h2</i> .
<code>mod_tp_dst:58866</code>	Traduce el puerto de destino de vuelta al valor original (58866) perteneciente a la sesión de <i>h2</i> .
<code>mod_dl_dst:00:00:00:00:00:02</code>	Modifica la dirección MAC de destino asignándole la MAC física del host <i>h2</i> .
<code>mod_dl_src:00:00:00:bb:bb:bb</code>	Reemplaza la dirección MAC de origen por la MAC interna (privada) del propio router.
<code>output:"s1-eth2"</code>	Expulsa el paquete finalizado a través del puerto 2 del switch para que llegue directamente a <i>h2</i> .

Ambos flujos tienen `idle_timeout=10`. El flujo entrante incluye el flag `OFPPF_SEND_FLOW_REM` para notificar al controlador al expirar y liberar el puerto NAT.

7.2. Múltiples clientes

Se realizó una prueba con dos clientes simultáneos (H2 y H3) conectándose al servidor H1 mediante **iperf UDP**, con el objetivo de verificar el soporte multi-host y la asignación independiente de puertos NAT.

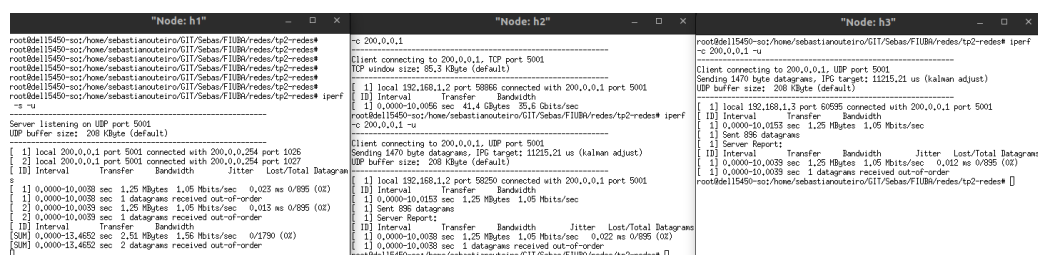


Figura 9: Prueba UDP con dos clientes simultáneos: H2 y H3 envían tráfico a H1 (puerto 5001). H1 recibe conexiones desde 200.0.0.254 con puertos 1026 y 1027.

Dado que la topología y el controlador permanecían levantados desde las pruebas TCP anteriores (donde ya se habían utilizado los puertos 1024 y 1025), el controlador asignó los puertos 1026 y 1027 para H2 y H3 respectivamente.

```

INFO:protonrouter:Paquete IP RECIBIDO: 192.168.1.2 + 200.0.0.1 | MAC: 00:00:00:00:00:02 + 00:00:00:bb:bb:bb | In Port: 2
INFO:protonrouter:Protocol 17 | src_port: 58250 + dst_port: 5001
INFO:protonrouter:MATCH: 192.168.1.2 pertenece a la red privada 192.168.1.0/24
INFO:protonrouter:ARP - Entrada expirada y eliminada: 200.0.0.1
INFO:protonrouter:ARP Request enviado: ¿Quién tiene 200.0.0.1? + port 1
INFO:protonrouter:ARP expirado para 200.0.0.1. Paquete encolado, ARP Request enviado.
INFO:protonrouter:ARP REPLY recibido de 200.0.0.1 (00:00:00:00:00:01) para 200.0.0.254 | In Port: 1
INFO:protonrouter:ARP - Entrada Agregada: 200.0.0.1 + 00:00:00:00:00:01
INFO:protonrouter:Procesando 1 paquete(s) pendiente(s) para 200.0.0.1
INFO:protonrouter:Puerto NAT asignado: 1026 | En uso: 1
INFO:protonrouter:PAT UDP: 192.168.1.2:58250 + 200.0.0.1:1026
INFO:protonrouter:ENVIANDO: 192.168.1.2 + 200.0.0.1 | NAT IP: 200.0.0.254 | MAC: 00:00:00:aa:aa:aa + 00:00:00:00:00:01 | Out Port: 1
INFO:protonrouter:ARP REQUEST recibido de 192.168.1.3 (00:00:00:00:00:03) para 192.168.1.254 | In Port: 3
INFO:protonrouter:ARP - Entrada Agregada: 192.168.1.3 + 00:00:00:00:00:03
INFO:protonrouter:ARP Reply enviado: 192.168.1.254 está en 00:00:00:bb:bb:bb + Out Port: 3
INFO:protonrouter:Paquete IP RECIBIDO: 192.168.1.3 + 200.0.0.1 | MAC: 00:00:00:00:00:03 + 00:00:00:bb:bb:bb | In Port: 3
INFO:protonrouter:Protocol 17 | src_port: 60595 + dst_port: 5001
INFO:protonrouter:MATCH: 192.168.1.3 pertenece a la red privada 192.168.1.0/24
INFO:protonrouter:Puerto NAT asignado: 1027 | En uso: 2
INFO:protonrouter:PAT UDP: 192.168.1.3:60595 + 200.0.0.1:1027
INFO:protonrouter:ENVIANDO: 192.168.1.3 + 200.0.0.1 | NAT IP: 200.0.0.254 | MAC: 00:00:00:aa:aa:aa + 00:00:00:00:00:01 | Out Port: 1
INFO:protonrouter:ARP REQUEST recibido de 192.168.1.2 (00:00:00:00:00:02) para 192.168.1.254 | In Port: 2
INFO:protonrouter:ARP - Entrada Agregada: 192.168.1.2 + 00:00:00:00:00:02
INFO:protonrouter:ARP Reply enviado: 192.168.1.254 está en 00:00:00:bb:bb:bb + Out Port: 2
INFO:protonrouter:ARP REQUEST recibido de 200.0.0.1 (00:00:00:00:00:01) para 200.0.0.254 | In Port: 1
INFO:protonrouter:ARP - Entrada Agregada: 200.0.0.1 + 00:00:00:00:00:01
INFO:protonrouter:ARP Reply enviado: 200.0.0.254 está en 00:00:00:aa:aa:aa + Out Port: 1
INFO:protonrouter:Puerto NAT liberado: 1026 | En uso: 1
INFO:protonrouter:Puerto NAT liberado: 1027 | En uso: 0

```

Figura 10: Logs del controlador POX durante la prueba UDP con dos clientes. Se aprecian la asignación de puertos NAT 1026 (H2) y 1027 (H3), y la liberación de ambos al finalizar.

Los eventos registrados por el controlador confirman el siguiente comportamiento:

- La entrada ARP de H1 había vencido (timeout de 10 s), por lo que el controlador emitió un nuevo ARP Request al recibir el paquete de H2.
- Como H2 ya conocía la MAC del router (entrada ARP vigente en el host), no fue necesario un nuevo ARP Request desde H2.
- Al llegar el paquete IP de H2, se asignó el puerto NAT 1026 y se instalaron los flujos correspondientes.
- H3 inició con un ARP Request hacia 192.168.1.254; el controlador respondió con la MAC privada. Como la MAC de H1 ya estaba en la tabla (aprendida al procesar el paquete de H2), el paquete de H3 se procesó inmediatamente asignando el puerto 1027.
- Al finalizar ambas conexiones, se liberaron los puertos 1026 y 1027, dejando el conteo en 0 puertos en uso.

Capturas Wireshark en H1 — Prueba UDP

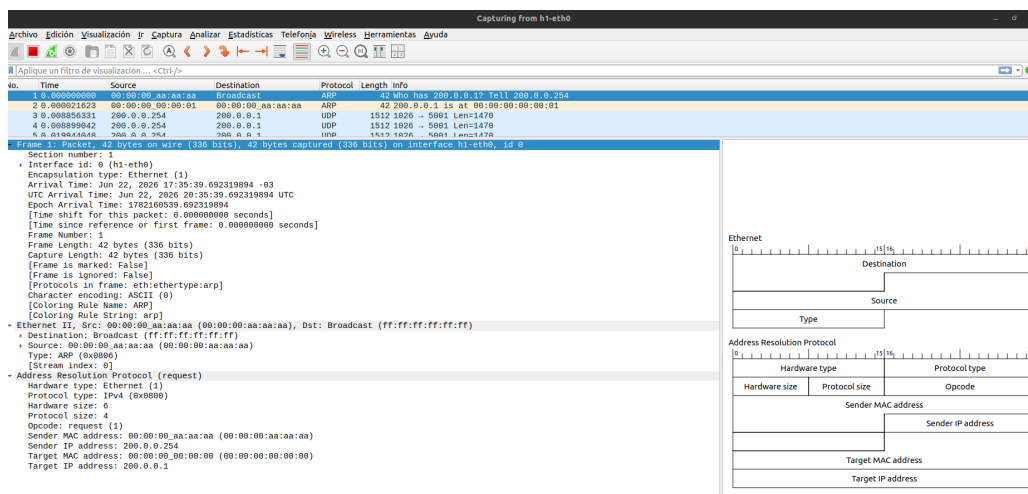


Figura 11: Wireshark en H1 (UDP): paquete 1, ARP Request desde la MAC pública del router buscando a H1; paquete 2, ARP Reply de H1.

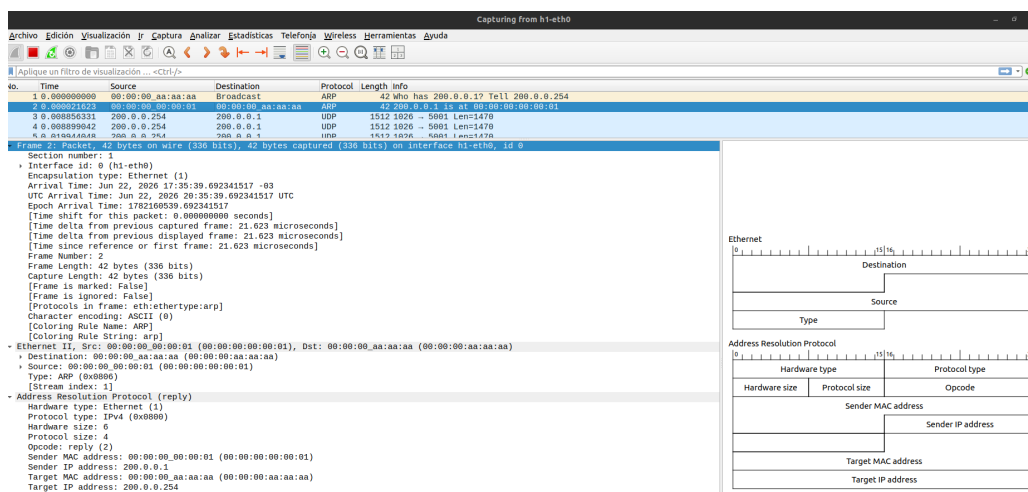


Figura 12: Wireshark en H1 (UDP): detalle del ARP Reply enviado por H1 al router.

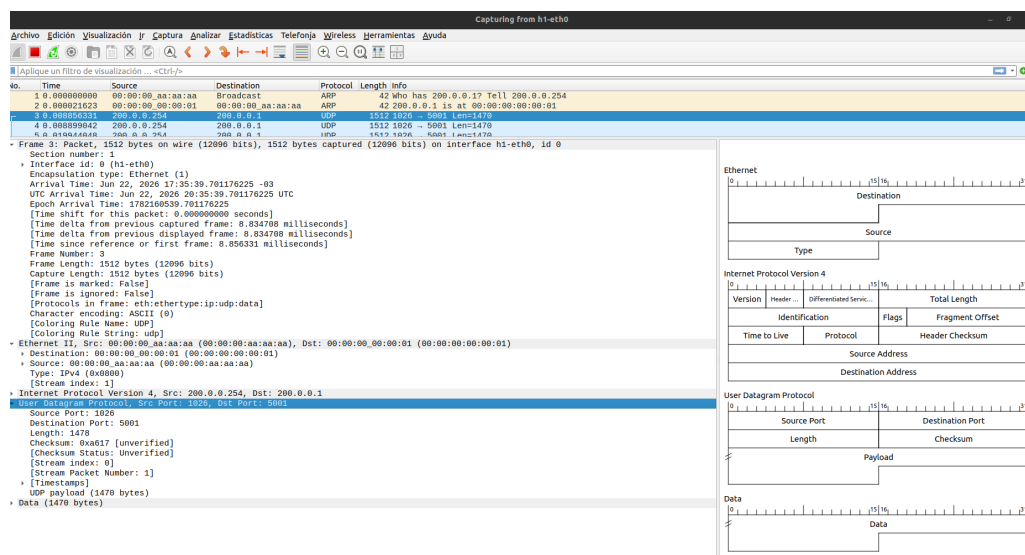


Figura 13: Wireshark en H1 (UDP): paquete 3, datagrama UDP desde 200.0.0.254:1026 hacia 200.0.0.1:5001, confirmando la traducción para el cliente H2.

Los timestamps en la tabla ARP del controlador cumplen la función de detectar cambios de MAC para una misma IP, garantizando la consistencia de las entradas ante posibles modificaciones en la red.

7.3. Diferentes tipos de tráfico

Las pruebas anteriores cubrieron los dos protocolos de transporte soportados por la implementación:

- **TCP** (Sección 7.1): verificado con `iperf` en modo TCP. Se comprobó el establecimiento del handshake (SYN / SYN-ACK / ACK), la traducción bidireccional de IP y puerto, y la liberación del puerto NAT al detectar la terminación de la conexión vía `FlowRemoved`.
- **UDP** (Sección 7.2): verificado con `iperf` en modo UDP con dos clientes simultáneos. Al tratarse de un protocolo sin conexión, la liberación del puerto depende exclusivamente del `idle_timeout` de 10 segundos. Se comprobó que ambos clientes reciben puertos NAT independientes y que el tráfico de retorno es enrutado al host privado correcto.

En ambos casos se verificó, mediante Wireshark y los logs del controlador, que:

- Solo los primeros paquetes de cada conexión (ARP y primer paquete IP) pasan por el controlador.
- El tráfico siguiente es procesado de forma autónoma por el switch, sin latencia adicional por intervención del controlador.
- Las traducciones de IP, puerto y MAC se aplican correctamente en ambas direcciones.

8. Conclusión

El trabajo práctico permitió implementar y verificar de forma integral un NAT saliente con traducción de puertos sobre una arquitectura SDN basada en OpenFlow y el controlador POX.

La decisión de delegar el estado de las conexiones activas a los flujos instalados en el switch, en lugar de mantener una tabla NAT explícita en el controlador, resultó ser efectiva: simplificó el diseño del plano de control y aprovechó de forma natural los mecanismos de expiración y notificación propios de OpenFlow para la liberación de puertos NAT.

Las pruebas realizadas cubrieron los escenarios principales de uso:

- **Conectividad básica TCP:** se verificó la traducción bidireccional de IP, puerto y MAC, el correcto funcionamiento del handshake TCP, y la liberación del puerto NAT al finalizar la conexión.
- **Múltiples clientes simultáneos (UDP):** se confirmó que el controlador asigna puertos NAT independientes a cada host privado y que el tráfico de retorno es enrutado al destinatario correcto, validando el soporte multi-host.
- **Eficiencia del plano de datos:** en todos los casos se verificó que únicamente los primeros paquetes de cada conexión (ARP y primer paquete IP) requieren intervención del controlador; el tráfico subsiguiente es procesado de forma autónoma por el switch.

Se identificó además la necesidad de mantener timestamps en la tabla ARP dinámica del controlador, para detectar y corregir posibles cambios de MAC asociados a una misma IP, garantizando la consistencia del reenvío ante modificaciones en la red.

En síntesis, la arquitectura SDN demostró ser un entorno adecuado para implementar funciones de red complejas como NAT/PAT de forma modular y verificable, separando claramente el plano de control del plano de datos y facilitando la inspección detallada del comportamiento mediante herramientas como Wireshark y los logs del controlador POX.